علوم کامپیوتر
نیم‌سال دوم ۰۴-۰۵
هوش مصنوعی

دانشکده ریاضی و آمار

زهرا چراغی، محمد ملائی، فاطمه سعیدی

گزارش پروژه منطق مرتبه اول: حل بازی مین‌روب

لینک گیت‌هاب پروژه: https://github.com/Elham-ch/AI-Projects/tree/main/AI26-PRJ5

۱ معرفی مسئله

در این پروژه، بازی Minesweeper به صورت یک مسئله استنتاج منطقی مدل شده است. صفحه بازی از خانه‌هایی تشکیل می‌شود که ممکن است مین داشته باشند یا امن باشند. با آشکار کردن یک خانه امن، عددی بین صفر تا هشت دیده می‌شود که تعداد مین‌های مجاور آن خانه را نشان می‌دهد. عامل باید فقط با استفاده از اطلاعاتی که تا همان لحظه آشکار شده‌اند، خانه‌های امن و مین‌ها را تشخیص دهد.

محیط بازی در فایل `src/minesweeper.py` قرار دارد و کلاس `MineSweeper` وظیفه تولید نقشه، نگهداری وضعیت خانه‌ها، اجرای عمل‌های `reveal` و `flag` و نمایش گرافیکی بازی را بر عهده دارد. نقطه شروع، خانه مرکزی صفحه است. سازنده محیط یک ناحیه سه در سه را در اطراف مرکز از مین خالی نگه می‌دارد؛ بنابراین مقدار خانه شروع همیشه صفر است.

در حل خودکار، گزینه `auto_flood_fill=False` استفاده شده است. در نتیجه حتی اگر یک خانه صفر آشکار شود، محیط به طور خودکار همسایه‌های آن را باز نمی‌کند. این تصمیم مهم است، زیرا عامل باید هر خانه جدید را از طریق پایگاه دانش خودش به دست آورد و اطلاعات پنهان محیط به طور ناخواسته وارد حافظه عامل نشود.

۲ ساختار کلی عامل

کد عامل در فایل `main.py` قرار دارد و از دو بخش اصلی تشکیل شده است:

- یک صفحه داخلی یا Shadow Board که فقط دانسته‌های عامل را نگهداری می‌کند؛
- یک پایگاه دانش pyDatalog که حقایق صفحه را دریافت می‌کند و با دو قانون اصلی، خانه‌های امن و مین‌ها را نتیجه می‌گیرد.

برای نمایش وضعیت خانه‌ها در صفحه داخلی از دو ثابت استفاده شده است:

```
AGENT_UNKNOWN = -1
AGENT_FLAGGED = -2
```

عددهای صفر تا هشت نیز مقدار واقعی خانه‌های آشکارشده‌اند.

۱.۲ همسایگی خانه‌ها

تابع `get_neighbors` همه همسایه‌های معتبر یک خانه را در هشت جهت برمی‌گرداند. شرط مرزی داخل همین تابع بررسی می‌شود؛ بنابراین خانه‌های گوشه سه همسایه، خانه‌های لبه پنج همسایه و خانه‌های داخلی هشت همسایه دارند.

```
for dr in (-1, 0, 1):
    for dc in (-1, 0, 1):
        if dr == 0 and dc == 0:
            continue
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            neighbors.append((nr, nc))
```

این رابطه در شروع بازی فقط یک بار به صورت حقایق `Adjacent(r,c,r2,c2)` در پایگاه دانش ثبت می‌شود و در طول بازی تغییر نمی‌کند.

۳ مدل‌سازی با منطق مرتبه اول

محمول‌های مورد استفاده در مدل منطقی عبارت‌اند از:

- `Adjacent(r,c,r2,c2)`: خانه دوم همسایه خانه اول است.
- `Value(r,c,n)`: خانه آشکارشده (r, c) عدد n را نشان می‌دهد.
- `FlaggedNeighborCount(r,c,f)`: تعداد همسایه‌های پرچم‌گذاری‌شده برابر f است.
- `HiddenNeighborCount(r,c,u)`: تعداد همسایه‌های ناشناخته و بدون پرچم برابر u است.
- `UnknownCell(r,c)`: خانه هنوز برای عامل ناشناخته است.
- `Safe(r,c)`: از حقایق فعلی نتیجه شده است که خانه امن است.
- `Mine(r,c)`: از حقایق فعلی نتیجه شده است که خانه مین دارد.
- `HiddenNeighbor(r,c,r2,c2)`: خانه دوم، همسایه ناشناخته یک سرنخ مرزی است.
- `FrontierClue(r,c,n,f)`: خانه آشکارشده‌ای در مرز دانش عامل با مقدار n و تعداد f پرچم مجاور است.

در این پیاده‌سازی، حقایق همسایگی ایستا هستند، اما مقدار خانه‌ها و شمارنده‌های اطراف آن‌ها پویا هستند. این جداسازی باعث می‌شود اطلاعات هندسی صفحه فقط یک بار ساخته شود و اطلاعات وابسته به وضعیت بازی در هر دور به‌روزرسانی شوند.

۱.۳ قانون امنیت

اگر عدد یک خانه آشکارشده با تعداد همسایه‌های پرچم‌گذاری‌شده آن برابر باشد، همه همسایه‌های ناشناخته باقی‌مانده امن هستند. شکل منطقی قانون چنین است:

$$\begin{aligned}&\text{Value}(r, c, n) \wedge \text{FlaggedCount}(r, c, n)\\ &\wedge\ \text{Adjacent}(r, c, r', c') \wedge \text{Unknown}(r', c')\\ &\qquad\Longrightarrow \text{Safe}(r', c')\end{aligned}$$

نسخه pyDatalog همین قانون در تابع `init_rules` تعریف شده است:

```
Safe(R2, C2) <= (
    Value(R, C, N)
    & FlaggedNeighborCount(R, C, N)
    & Adjacent(R, C, R2, C2)
    & UnknownCell(R2, C2)
)
```

برای مثال اگر خانه‌ای مقدار دو داشته باشد و دقیقا دو همسایه آن پرچم‌گذاری شده باشند، هیچ مین دیگری نمی‌تواند در میان همسایه‌های ناشناخته آن وجود داشته باشد.

۲.۳ قانون خطر

فرض می‌کنیم عدد خانه آشکارشده برابر n، تعداد پرچم‌های مجاور برابر f و تعداد خانه‌های ناشناخته بدون پرچم برابر u باشد. اگر

$$n = f + u,$$

آنگاه همه u خانه ناشناخته باید مین باشند. شکل منطقی قانون به صورت زیر است:

$$\begin{aligned}&\text{Value}(r, c, n) \wedge \text{FlaggedCount}(r, c, f) \wedge \text{HiddenCount}(r, c, u)\\ &\wedge\ n = f + u \wedge u > 0 \wedge \text{Adjacent}(r, c, r', c') \wedge \text{Unknown}(r', c')\\ &\qquad\Longrightarrow \text{Mine}(r', c')\end{aligned}$$

در کد، این قاعده به شکل زیر نوشته شده است:

```
Mine(R2, C2) <= (
    Value(R, C, N)
    & FlaggedNeighborCount(R, C, F)
    & HiddenNeighborCount(R, C, U)
    & (N == F + U)
    & (U > 0)
```

```
    & Adjacent(R, C, R2, C2)
    & UnknownCell(R2, C2)
)
```

وجود شرط U > 0 از تولید نتیجه‌های بی‌اثر برای خانه‌هایی جلوگیری می‌کند که دیگر همسایه ناشناخته ندارند.

۴ ساخت و به‌روزرسانی پایگاه دانش

تابع update_knowledge_base در ابتدای هر دور اجرا می‌شود. ابتدا تمام حقایق پویای دور قبل با استفاده از فهرست dynamic_facts حذف می‌شوند. سپس صفحه داخلی پیمایش می‌شود و برای هر خانه، حقایق جدید متناسب با وضعیت فعلی ساخته می‌شوند.

برای یک خانه آشکارشده، تعداد پرچم‌ها و خانه‌های ناشناخته فقط از روی صفحه داخلی عامل محاسبه می‌شود:

```
flagged_count = sum(
    1 for nr, nc in neighbors
    if agent_board[(nr, nc)] == AGENT_FLAGGED
)
hidden_count = sum(
    1 for nr, nc in neighbors
    if agent_board[(nr, nc)] == AGENT_UNKNOWN
)
```

اگر یک خانه آشکارشده هیچ همسایه ناشناخته‌ای نداشته باشد، دیگر نمی‌تواند در قانون امنیت یا خطر نتیجه تازه‌ای تولید کند. به همین دلیل برای چنین خانه‌ای حقایق Value، FlaggedNeighborCount و HiddenNeighborCount دوباره ثبت نمی‌شوند. بنابراین اندازه بخش پویای پایگاه دانش به جای همه خانه‌های آشکارشده، بیشتر به اندازه مرز فعلی دانش عامل وابسته می‌ماند.

بعد از ساخته شدن حقایق، تابع query_solver دو پرس‌وجوی Safe(R,C) و Mine(R,C) را اجرا می‌کند و هم‌زمان نتیجه استنتاج چندسرنخی را نیز به آن‌ها می‌افزاید. پاسخ‌های هر دو روش در دو فهرست مختصات ادغام می‌شوند و نتیجه تکراری در فهرست قرار نمی‌گیرد.

۵ استنتاج چندسرنخی روی مرز

دو قانون اصلی هر بار فقط یک سرنخ را بررسی می‌کنند. در بعضی وضعیت‌ها هیچ‌کدام از آن‌ها به تنهایی کافی نیستند، اما مقایسه دو سرنخ هم‌پوشان یک حرکت قطعی تولید می‌کند. برای هر سرنخ مرزی، قید زیر ساخته می‌شود:

$$\sum_{x \in U_i} x = n_i - f_i,$$

که در آن U_i مجموعه همسایه‌های ناشناخته، n_i مقدار سرنخ و f_i تعداد پرچم‌های مجاور است. عضویت خانه‌ها در U_i از طریق دو قاعده pyDatalog زیر به دست می‌آید:

```
HiddenNeighbor(R, C, R2, C2) <= (
    Adjacent(R, C, R2, C2)
    & UnknownCell(R2, C2)
)
FrontierClue(R, C, N, F) <= (
    Value(R, C, N)
    & FlaggedNeighborCount(R, C, F)
)
```

سپس برای دو قید هم‌پوشان i و j، بخش مشترک حذف می‌شود:

$$\sum_{x \in U_i \setminus U_j} x - \sum_{x \in U_j \setminus U_i} x = (n_i - f_i) - (n_j - f_j).$$

اگر اختلاف سمت راست به یکی از کران‌های ممکن برسد، خانه‌های یک سمت همگی مین و خانه‌های سمت دیگر همگی امن خواهند بود. این روش هم کاهش زیرمجموعه‌ای و هم الگوهایی مانند 1-2-1 را پوشش می‌دهد. استنتاج تا رسیدن به نقطه ثابت ادامه پیدا می‌کند تا نتیجه یک مقایسه بتواند قیدهای دیگر را نیز ساده کند.

تابع query_solver نتیجه این مرحله را همیشه با خروجی دو قانون مستقیم ادغام می‌کند. بنابراین حلقه اصلی نیازی به مسیر جداگانه یا کد تکراری برای اعمال حرکت‌های چندسرنخی ندارد.

۶ حلقه اصلی بازی

مراحل عامل در هر دور به ترتیب زیر هستند:

۱. بازسازی حقایق پویای پایگاه دانش از روی صفحه داخلی؛

۲. پرس‌وجوی خانه‌های امن و مین‌ها و ادغام نتیجه استنتاج مستقیم و چندسرنخی؛

۳. آشکار کردن همه خانه‌هایی که امن نتیجه شده‌اند؛

۴. پرچم‌گذاری خانه‌هایی که مین نتیجه شده‌اند؛

۵. انتخاب یک حدس فقط وقتی هیچ حرکت قطعی تولید نشده باشد؛

۶. نمایش وضعیت جدید بازی و آغاز دور بعد.

خانه‌های امن پیش از مین‌ها اعمال می‌شوند. هر عمل نیز فقط در صورتی انجام می‌شود که خانه هنوز در صفحه داخلی ناشناخته باشد. نتیجه‌های مستقیم و چندسرنخی پیش از بازگشت از `query_solver` بدون تکرار ادغام می‌شوند؛ در نتیجه فقط یک حلقه برای آشکار کردن و یک حلقه برای پرچم‌گذاری لازم است و عمل تکراری روی محیط اجرا نمی‌شود.
نقطه شروع در سازنده محیط آشکار شده است. از آنجا که صورت پروژه تضمین می‌کند مقدار این خانه صفر است، عامل مقدار صفر را در صفحه داخلی ثبت می‌کند و استنتاج را از همسایه‌های آن آغاز می‌کند.

۷ مدیریت بن‌بست منطقی

حتی پس از اجرای قوانین مستقیم و استنتاج چندسرنخی، ممکن است چند آرایش متفاوت از مین‌ها با همه حقایق فعلی سازگار باشند و هیچ خانه‌ای به طور قطعی امن یا مین نباشد. در این حالت تابع `get_safest_guess` برای خانه‌های مرزی یک تخمین ساده از خطر می‌سازد.
برای هر خانه آشکارشده i با عدد n_i، تعداد پرچم f_i و تعداد همسایه ناشناخته u_i، خطر محلی هر همسایه ناشناخته به صورت زیر در نظر گرفته می‌شود:

$$p_i = \frac{n_i - f_i}{u_i}.$$

اگر یک خانه ناشناخته x کنار چند سرنخ آشکارشده باشد، میانگین تخمین‌های محلی آن‌ها محاسبه می‌شود:

$$\widehat{P}(\mathrm{Mine}(x)) = \frac{1}{|C(x)|} \sum_{i \in C(x)} p_i,$$

که $C(x)$ مجموعه سرنخ‌های مجاور خانه x است. عامل خانه‌ای را انتخاب می‌کند که کمترین میانگین را داشته باشد. اگر هیچ سرنخی برای ساخت این تخمین وجود نداشته باشد، یک خانه ناشناخته به صورت تصادفی انتخاب می‌شود.
این مقدار یک احتمال دقیق مشترک نیست، زیرا وابستگی قیدهای هم‌پوشان را مدل نمی‌کند. با این حال، از حدس کاملا تصادفی بهتر از اطلاعات موجود استفاده می‌کند و فقط پس از تمام شدن حرکت‌های قطعی فعال می‌شود.

۸ آزمون قوانین منطقی

پیش از اجرای سناریوهای کامل، سه حالت کوچک برای بررسی مستقل قوانین ساخته شد:

۱. خانه مرکزی مقدار یک داشت، یکی از همسایه‌های آن پرچم‌گذاری شده بود و هفت همسایه دیگر ناشناخته بودند. پرس‌وجو دقیقا هر هفت خانه باقی‌مانده را امن برگرداند و هیچ مین جدیدی نتیجه نشد.

۲. خانه مرکزی مقدار هشت داشت و هر هشت همسایه آن ناشناخته بودند. پرس‌وجو دقیقا هر هشت همسایه را مین برگرداند و هیچ خانه امنی نتیجه نشد.

۳. سه سرنخ متوالی با الگوی 1-2-1 در لبه صفحه قرار گرفتند. استنتاج چندسرنخی خانه میانی مرز را امن و دو خانه کناری را مین نتیجه گرفت؛ در حالی که بررسی جداگانه هر سرنخ برای این نتیجه کافی نبود.

دو آزمون نخست نشان می‌دهند شرط‌های شمارشی قانون امنیت و قانون خطر به همان صورتی که در مدل نوشته شده‌اند در موتور استنتاج اجرا می‌شوند. آزمون سوم نیز ادغام قیدهای هم‌پوشان و بازگشت نتیجه آن از `query_solver` را بررسی می‌کند.

۹ نحوه ارزیابی

چهار سناریوی مشخص‌شده در صورت پروژه اجرا شدند. برای اینکه زمان به‌دست‌آمده مربوط به استنتاج باشد، اجرای آزمایشی به صورت headless انجام شد و فراخوانی نمایش گرافیکی و تاخیر ثابت `time.sleep(0.2)` از اندازه‌گیری کنار گذاشته شد. خود منطق عامل، ترتیب اعمال حرکت‌ها و راهبرد حدس بدون تغییر باقی ماندند.

۱۰ نتایج اجرا

ابعاد	مین	Seed	نتیجه	پیشرفت	دورها	امن منطقی	مین منطقی	زمان
9×9	۹	۹۹	برد	100%	۸	۷۱	۸	$0.170\,s$
15×15	۳۵	۴۲	برد	100%	۱۰	۱۸۹	۳۵	$0.574\,s$
20×20	۷۵	۱۲۳	باخت پس از ۱۱ حدس	99.08%	۳۳	۳۱۰	۷۱	$0.764\,s$
45×80	۲۰۰	-1	برد	100%	۴۰	۳۳۹۹	۱۹۶	$8.407\,s$

در سناریوی ساده و استاندارد، عامل بدون هیچ حدسی صفحه را کامل حل کرد. تعداد خانه‌های امن منطقی در این دو اجرا به ترتیب ۷۱ و ۱۸۹ است؛ با احتساب خانه شروع، این عددها دقیقا برابر تعداد کل خانه‌های امن صفحه می‌شوند. در سناریوی استاندارد هر ۳۵ مین پرچم‌گذاری شد. در سناریوی ساده، بازی پس از آشکار شدن آخرین خانه امن و پیش از نیاز به پرچم‌گذاری آخرین مین با پیروزی پایان یافت؛ به همین دلیل تعداد مین منطقی ۸ ثبت شده است.

در سناریوی چالشی، عامل ابتدا با استنتاج قطعی بخش عمده صفحه را حل کرد، اما در مجموع ۱۱ بار به بن‌بست رسید. حدس یازدهم روی یک مین قرار گرفت و بازی با پیشرفت 99.08% تمام شد.

در سناریوی بزرگ، عامل همه ۳۴۰۰ خانه امن را بدون حدس آشکار کرد و بازی با پیروزی پایان یافت. از آنجا که شرط پیروزی با آشکار شدن همه خانه‌های امن برقرار می‌شود، حلقه پیش از پرچم‌گذاری چهار مین باقی‌مانده متوقف شد و تعداد مین منطقی ۱۹۶ ثبت شد. این آزمایش نشان داد پیاده‌سازی روی صفحه 80×45 بدون کرش اجرا می‌شود. زمان تقریبا ۴.۸ ثانیه نیز اثر حذف سرنخ‌های کاملا حل‌شده از پایگاه دانش پویا را در مقایسه با اجرای قبلی نشان می‌دهد.

۱۱ تحلیل پیاده‌سازی

۱.۱۱ دلیل موفقیت استنتاج منطقی

قانون امنیت و قانون خطر دو حالت حدی یک قید شمارشی را پوشش می‌دهند. اگر همه مین‌های لازم قبلا با پرچم مشخص شده باشند، بقیه همسایه‌ها امن هستند. اگر تعداد مین باقی‌مانده دقیقا با تعداد خانه‌های ناشناخته برابر باشد، همه آن خانه‌ها مین هستند. مقایسه قیدهای دو سرنخ هم‌پوشان نیز حالت‌هایی را پوشش می‌دهد که هیچ سرنخی به تنهایی به یکی از این دو حد نرسیده است. اجرای دسته‌ای و ادغام نتیجه‌ها باعث می‌شود در هر دور چندین خانه آشکار یا پرچم‌گذاری شوند و حقایق جدید برای دور بعد تولید شوند.

۲.۱۱ حفظ مرز اطلاعات عامل و محیط

عامل برای تصمیم‌گیری فقط از agent_board استفاده می‌کند. آرایه `_grid_values` که جای واقعی مین‌ها را نگهداری می‌کند، در کد حل‌کننده خوانده نمی‌شود. مقدار هر خانه فقط از خروجی `reveal` و وضعیت پرچم فقط از عمل خود عامل وارد حافظه می‌شود. بنابراین نتیجه‌های منطقی بر پایه اطلاعات قابل مشاهده بازی هستند.

۳.۱۱ هزینه اجرای موتور استنتاج

تعداد حقایق همسایگی با تعداد خانه‌ها رشد خطی دارد و برای بیشتر خانه‌های داخلی هشت رابطه ساخته می‌شود. در هر دور هنوز صفحه داخلی پیمایش می‌شود، اما حقایق شمارشی فقط برای سرنخ‌هایی ثبت می‌شوند که حداقل یک همسایه ناشناخته دارند. حذف سرنخ‌های حل‌شده از پایگاه دانش باعث می‌شود هزینه پرس‌وجو در مراحل پایانی بازی با تعداد کل خانه‌های آشکارشده رشد نکند. با این حال، بازسازی حقایق ناشناخته و پیمایش کامل صفحه همچنان در صفحه‌های بسیار بزرگ هزینه دارد.

۱۲ محدودیت‌ها و پیشنهادهای بهبود

مهم‌ترین محدودیت‌ها و مسیرهای بهبود نسخه فعلی عبارت‌اند از:

- استنتاج چندسرنخی فعلی قیدها را به صورت زوجی مقایسه می‌کند. حل هم‌زمان یک مؤلفه کامل از قیدهای مرزی می‌تواند نتیجه‌های عمیق‌تری تولید کند، اما پیچیدگی بیشتری دارد.
- میانگین خطر محلی یک تخمین ساده است و سازگاری هم‌زمان همه قیدها را بررسی نمی‌کند. شمارش مدل‌های سازگار یا حل مؤلفه‌های مرزی می‌تواند احتمال دقیق‌تری تولید کند.
- با وجود حذف سرنخ‌های کاملا حل‌شده، حقایق پویا در هر دور بازسازی می‌شوند. به‌روزرسانی افزایشی فقط برای خانه‌هایی که وضعیتشان تغییر کرده است می‌تواند هزینه صفحه‌های بزرگ را بیشتر کاهش دهد.
- در صورت برابر بودن خطر چند خانه، انتخاب فعلی به ترتیب پیمایش دیکشنری وابسته است. می‌توان معیارهای تکمیلی مانند بیشترین تعداد سرنخ مجاور یا بیشترین اطلاعات مورد انتظار را برای شکستن تساوی استفاده کرد.